



PRESIDENCY UNIVERSITY

Department of Computer Science and Engineering

CSE322

Microprocessors Based System Design Laboratory

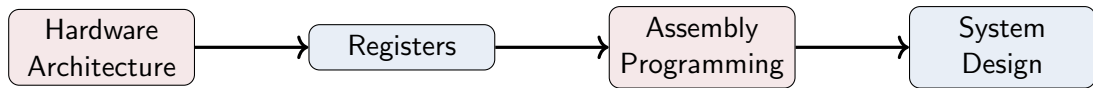
**8086 Architecture, Registers
& Memory Organization**

Learning Goals

At the end of this lecture, students will be able to:

- ➊ Explain the evolution and features of Intel 8086.
- ➋ Identify the major blocks of 8086 architecture.
- ➌ Explain BIU and EU operations.
- ➍ Describe all important 8086 registers.
- ➎ Understand memory segmentation.
- ➏ Calculate physical addresses in 8086.
- ➐ Relate architecture concepts with assembly programming.

Why Do We Study 8086 Architecture?



The architecture explains how instructions, data, memory and hardware components work together.

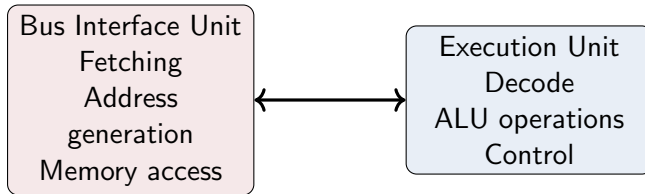
8086 Overview

Intel 8086 is a 16-bit CISC microprocessor introduced in 1978. It became the foundation of the x86 processor family.

- 16-bit ALU
- 16-bit registers
- 20-bit address bus
- 16-bit data bus
- Supports 1 MB memory

8086 CPU
16-bit Processing
20-bit Addressing

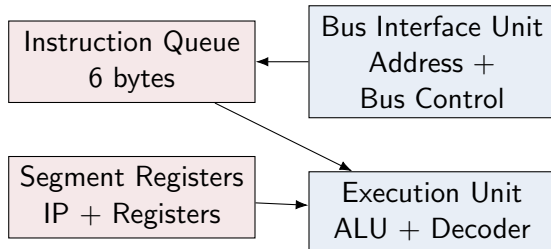
Basic 8086 Organization



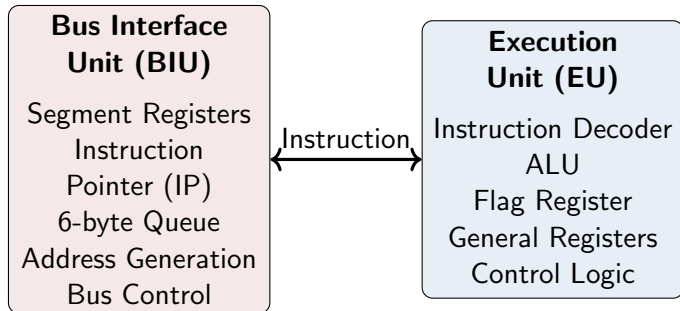
Main Idea

8086 separates instruction fetching and instruction execution to improve speed.

8086 Internal Architecture



8086 Internal Architecture: Complete View



8086 Internal Design Philosophy

The 8086 is divided into two independent units so that fetching and execution can happen simultaneously.

Internal Architecture: Role of BIU Components

Segment Registers

- CS stores Code Segment address.
- DS stores Data Segment address.
- SS stores Stack Segment address.
- ES provides additional data segment.

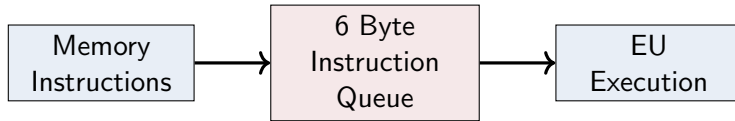
Instruction Pointer (IP)

- Holds offset address of next instruction.
- Works together with CS.
- Generates instruction address.

Example

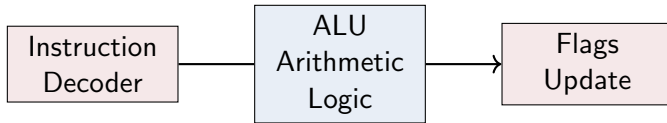
If CS = 2000H and IP = 0100H, next instruction is fetched from 2000:0100.

Internal Architecture: Instruction Queue



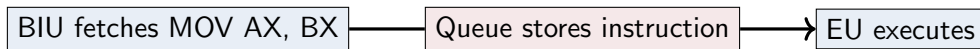
- BIU continuously fetches instruction bytes.
- Queue stores upcoming instructions temporarily.
- EU removes instructions from queue for execution.
- This reduces CPU waiting time.

Internal Architecture: Role of EU Components



- Decoder understands the instruction opcode.
- ALU performs arithmetic and logical operations.
- Flag register stores result conditions.
- Control logic coordinates all EU operations.

How BIU and EU Work Together: Example



- 1 BIU calculates instruction address using CS:IP.
- 2 Instruction bytes are fetched from memory.
- 3 Bytes are placed into the 6-byte queue.
- 4 EU decodes and executes the instruction.

Bus Interface Unit (BIU)

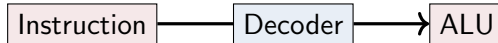
Responsibilities

- Generates 20-bit physical memory addresses.
- Fetches instructions from memory.
- Stores prefetched instructions in queue.
- Controls memory and I/O transfers.

BIU = Communication between CPU and outside world

Execution Unit (EU)

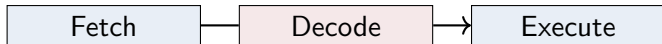
- Receives instructions from instruction queue.
- Decodes instruction operation.
- Performs arithmetic and logical operations.
- Updates flags after execution.



Instruction Pipelining in 8086: Basic Idea

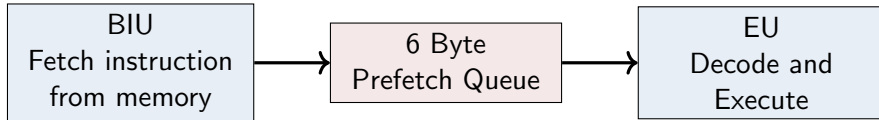
Why Pipelining?

A traditional processor completes one instruction before starting the next one. This creates idle time because fetching and execution cannot happen together.



- 8086 improves speed by overlapping instruction fetch and execution.
- BIU fetches the next instruction while EU executes the current instruction.
- This technique is called instruction pipelining.

8086 Pipeline Operation: BIU and EU Cooperation



- BIU and EU work independently.
- The queue acts as a buffer between fetching and execution.
- When EU needs a new instruction, it takes bytes from the queue.

Instruction Pipeline Timing Example

Clock	BIU Activity	EU Activity
T1	Fetch Instruction 1	Idle
T2	Fetch Instruction 2	Decode Instruction 1
T3	Fetch Instruction 3	Execute Instruction 1
T4	Fetch Instruction 4	Execute Instruction 2
T5	Fetch Instruction 5	Execute Instruction 3

Important Observation

Fetching and execution occur at the same time. Therefore, CPU utilization increases.

Without Pipeline vs With Pipeline

Without Pipeline

I1: Fetch → Decode → Execute
I2: Fetch → Decode → Execute
I3: Fetch → Decode → Execute

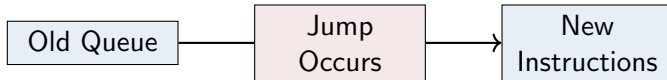
With 8086 Pipeline

Cycle 1: I1 Fetch
Cycle 2: I1 Execute + I2 Fetch
Cycle 3: I1 Complete + I2 Execute + I3 Fetch

- Multiple instructions stay in different stages simultaneously.
- This improves instruction throughput.

Pipeline Limitations in 8086

- **Jump Instructions:** The queue may contain wrong instructions after a branch.
- **Interrupts:** Current queue contents may need to be discarded.
- **Memory Delay:** Slow memory access can reduce pipeline efficiency.



8086 Register Organization

General Purpose
AX BX CX DX

Segment
CS DS SS ES

Pointer & Index
SP BP SI DI

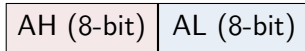
General Purpose Registers

Register	Name	Common Use
AX	Accumulator	Arithmetic operations
BX	Base Register	Memory addressing
CX	Count Register	Loop counting
DX	Data Register	Multiplication/I/O

Accumulator Register AX

AX is a 16-bit register divided into two 8-bit registers:

$$AX = AH + AL$$



Example:

MOV AX, 1234H

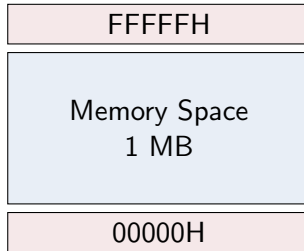
Segment Registers

8086 divides memory into segments.



- CS stores program instructions.
- DS stores data.
- SS manages stack.
- ES supports additional data.

Memory Organization of 8086



Why Segmentation?

- 8086 registers are 16-bit.
- Memory address is 20-bit.
- Segmentation combines two values to create addresses.

$$\text{Physical Address} = \text{Segment} \times 16 + \text{Offset}$$

Physical Address Calculation Example

Given:

$$CS = 2000H, \quad IP = 4500H$$

Calculation:

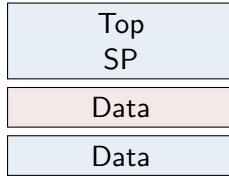
$$2000H \times 10H = 20000H$$

$$20000H + 4500H = 24500H$$

Therefore:

$$\boxed{\text{Physical Address} = 24500H}$$

Stack Organization



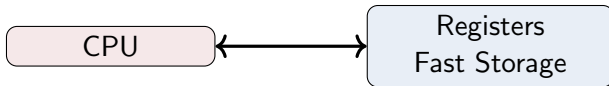
Stack follows LIFO principle:

Last In First Out

What is a Register?

Definition

A register is a small, high-speed storage location inside the CPU used to temporarily hold data, instructions, addresses and intermediate results during processing.



- Registers are faster than RAM.
- 8086 contains several 16-bit registers.
- Different registers have different purposes.

Why Does CPU Need Registers?



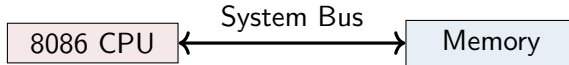
CPU cannot directly perform operations on slow memory. Data is first moved into registers, then processed by the ALU.

Memory $\rightarrow$ Register $\rightarrow$ ALU $\rightarrow$ Register $\rightarrow$ Memory

Understanding Buses in 8086

Bus Concept

A bus is a collection of electrical lines used for communication between CPU, memory and peripheral devices.



Three important buses:

- Address Bus: Selects memory location.
- Data Bus: Transfers data.
- Control Bus: Carries read/write and control signals.

8086 Bus Size and Memory Capacity

Bus	Size	Purpose
Address Bus	20 bits	Select memory location
Data Bus	16 bits	Transfer 16-bit data
Control Bus	Multiple signals	Control operations

A 20-bit address bus can generate:

$$2^{20} = 1,048,576 \text{ locations} = 1MB$$

Therefore, 8086 can access 1 MB memory space.

Why Memory Segmentation is Required?

A 16-bit register can represent only:

$$2^{16} = 65536 \text{ bytes} = 64KB$$

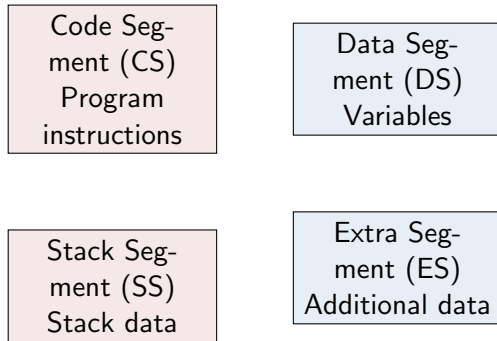
But 8086 needs to access 1 MB memory.



Segmentation combines two 16-bit values to create a 20-bit address.

8086 Memory Segments

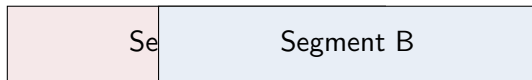
8086 divides memory into logical segments:



Each segment can have maximum size of 64 KB.

Segment Overlapping Concept

Segments do not have fixed boundaries. They may overlap with each other.



The same physical memory location can belong to multiple logical segments.

Example:

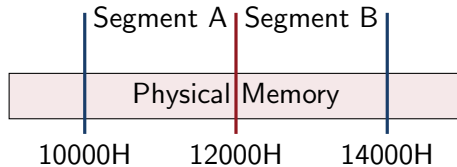
$$1000H : 0020H = 1002H : 0000H$$

Both generate the same physical address.

Segment Overlapping Concept: Why Does It Happen?

Basic Idea

In 8086, memory is divided into logical segments. Each segment can start at a different physical address, but segments are allowed to overlap with each other.



- A segment does not have a fixed physical area.
- Two segments may share some common memory locations.
- This is called **segment overlapping**.

Why Does 8086 Allow Segment Overlapping?

Reason 1: Limited Register Size

8086 registers are only 16-bit, but memory addresses are 20-bit. Segmentation solves this limitation.

Reason 2: Flexible Memory Use

Programs can create different logical views of the same physical memory area.

Important Concept

The segment register stores the starting point. The offset selects a location inside that segment.

Detailed Physical Address Calculation

Formula:

$$\text{Physical Address} = (\text{Segment} \times 10H) + \text{Offset}$$

Example:

Segment = 1234H

Offset = 5678H

$$1234H \times 10H = 12340H$$

$$12340H + 5678H = 179B8H$$

Important

Segment value is shifted left by 4 bits before adding offset.

Segment Overlapping: Numerical Example

Consider two logical addresses:

Address 1

$1000H : 0020H$

$$1000H \times 16 + 0020H$$

$$10000H + 0020H = 10020H$$

Address 2

$1002H : 0000H$

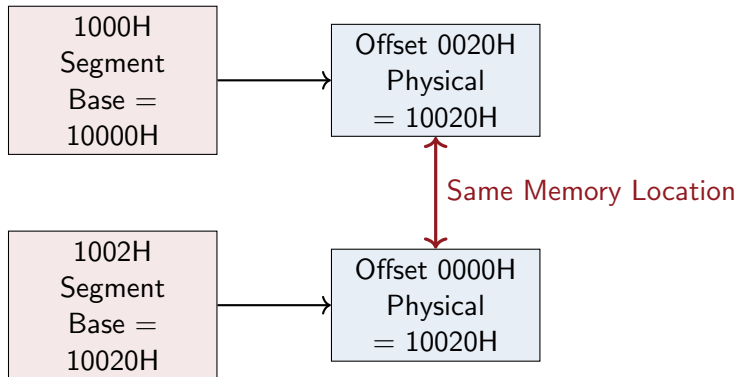
$$1002H \times 16 + 0000H$$

$$10020H + 0000H = 10020H$$

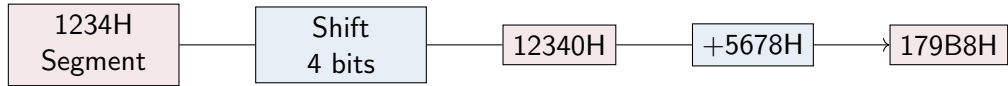
Observation

Two different segment:offset combinations generate the same physical address.

Visualizing Overlapping Addresses



Address Calculation Visualization



This mechanism allows a 16-bit CPU to address 20-bit memory locations.

Assembly Language Connection

Architecture directly affects programming.

Example:

MOV AX,05H

Meaning:

- Put value 5 into accumulator.
- AX register stores the data.
- EU executes the instruction.

Laboratory Perspective

Students should be able to:

- ① Identify registers used by instructions.
- ② Calculate memory addresses.
- ③ Trace instruction execution.
- ④ Understand register contents during debugging.

Next Lecture

8086 Instruction Set and Assembly Programming Environment

Summary

- 8086 is a 16-bit CISC microprocessor.
- BIU and EU divide processor activities.
- Registers provide fast temporary storage.
- Segmentation allows 1 MB memory addressing.
- Physical address = Segment $\times 16$ + Offset.



Thank You

Questions?

Afzal Hossan
Presidency University